EXPLAIN_AND_UNDERSTAND_PROJECT — A Narrative Walkthrough of the Financial RAG System

> **Who this is for.** An engineer who must fully understand this repository by reading this document **next to the code it cites**.
>
> **How this was produced.** An active whole-repository discovery scan (symbol dumps plus direct reads of every subsystem) was run before a single word was written. Every claim below is anchored to real code with a sparse `file:line` reference. Where a commonly-repeated claim turned out to be false in the code, it is called out inline as an **Audit note**.
>
> **Path reality.** The brief's paths (`src/3_reranking/`, `src/evaluation/`, `src/app/analytics.py`, `dashboard/`, `streamlit_dashboard.py`) do not exist in this repository. The real layout is `src/4_retrieval/`, root-level `evaluation/`, `app/api/analytics.py`, `frontend/`, and `app/ui/streamlit_app.py`. Real paths are used throughout this document.

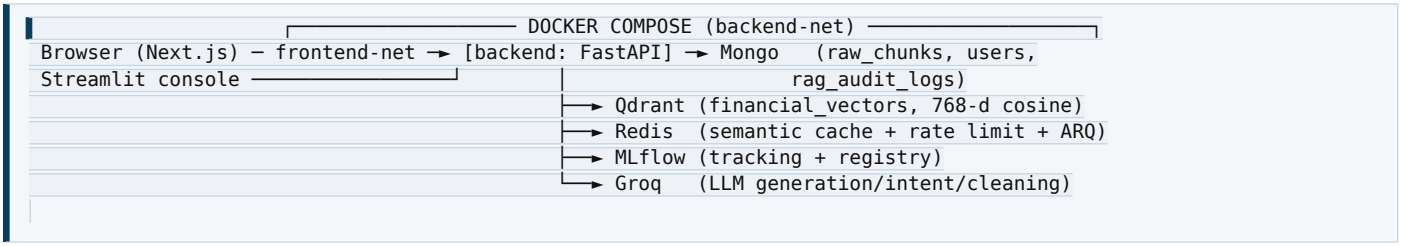
This document has exactly three parts, in the order you should read them: first the **end-to-end system flow**, which shows how a single document and a single question travel through the entire machine; then the **detailed implementation walkthrough**, component by component, in narrative form; then a **lightweight Docker and frontend overview**.

---

1. THE END-TO-END SYSTEM FLOW (Read This First)

1.1 The shape of the machine

Everything runs inside one Docker Compose stack split across two networks. The Next.js browser app speaks to a FastAPI backend over `frontend-net`; the backend alone sits on `backend-net`, where it talks to MongoDB (document store), Qdrant (vector store), Redis (cache, rate limits, job queues), MLflow (experiment tracking and model registry), and outward to Groq cloud models for generation, intent classification, and cleaning. The single load-bearing invariant the whole design serves is: **a number that appears in an answer must be traceable to extracted evidence — otherwise the system refuses, or an enforcement layer swaps the answer for a safe fallback.**



Two flows matter: the **document's life** (ingestion, run once per filing) and the **question's life** (querying, run on every user turn). Both are narrated below.

1.2 A document's life

A filing enters the system through an authenticated, admin-only upload endpoint — there is no SEC downloader in this repository; filings are seeded manually with `docker compose cp` and then indexed. The backend detects the file type itself: SGML and HTML filings are parsed by a purpose-written parser, PDFs by LlamaParse with a pypdf fallback, and DOCX files by LlamaParse with a python-docx fallback. Tables are the hard part: the HTML table parser flattens every table into Markdown and simultaneously plants a `%%TABLE_N%%` placeholder token at the table's position in the text, so that downstream code can always find the **unflattened** table again when a question is about numbers.

With Markdown-plus-placeholders in hand, the cleaning stage runs eight financial-safe normalization steps — the kind of transformations that must never change a figure's value (dollar signs, thousands separators, unicode dashes, whitespace) — while protecting already-extracted table fragments behind `§FIN{n}§` sentinels so the normalizer cannot mangle them. Metadata extraction then classifies the document by walking 20 regex patterns for SEC section headers (Item 7, Item 8, and their variants), and deliberately returns **no** document type when the accession number is ambiguous rather than guessing.

Chunking is hybrid and tiered: prose goes to token-based chunks, tables to their own boundary-respecting chunks, and each chunk receives the canonical `chunk_id` built as `ticker_type_md5[:12]_index:04d`. That identifier, not the chunk's text or position, is the system's primary key everywhere. Embeddings are computed by a single warm

EmbeddingEngine running `nomic-ai/nomic-embed-text-v1.5` at 768 dimensions on CUDA, and the write is **dual**: MongoDB receives the full chunk record (unique index on `chunk_id`, so a re-ingest upserts instead of duplicating), while Qdrant receives a point whose ID is a deterministic `uuid5(NAMESPACE_DNS, chunk_id)` carrying only seven payload fields — the raw text never reaches the vector store. Re-ingesting a document is therefore safe by construction: a verify-then-delete `safe_replacement` flow confirms the new chunks exist before removing old ones, and a read-only `reconcile_identity` pass can detect drift between the two stores without mutating either.

1.3 A question's life (synchronous chat)

This is the hot path. Each request passes through the API surface, where middleware resolves a rate-limit key (`user:` for authenticated callers, `ip:` otherwise, with a token bucket on chat at ten requests per minute — `app/api/rate_limiter.py:118`) and the endpoint restores the caller's conversation memory into the pipeline's session object (`app/api/main.py:854`). The orchestrator then runs its stages in a fixed order (`src/pipeline.py:1621`, `query()`), opening an MLflow run as it goes:

1. **Cache lookup.** The semantic cache is consulted first — but only if the previous answer's guardrail verdict was `passed` (`src/pipeline.py:1680`). The lookup itself walks three tiers: an exact SHA-256 key of the normalized question, then cosine similarity of stored embeddings against the new question, then a cross-encoder rerank gate. A hit returns immediately, skipping retrieval and generation entirely.
2. **Pre-retrieval routing** (disabled by default: `ENABLE_PRE_RETRIEVAL=False`, dormant at `src/pipeline.py:1727`). When on, a single LLM call classifies the question for safety, action, ticker, and year and can rewrite or expand it; a rule-based router is the fallback, and a security-violation verdict halts the turn with a fixed refusal.
3. **Hybrid retrieval.** Dense vector search and BM25 keyword search are fused with Reciprocal Rank Fusion (k=60), with a token-overlap gate and a corpus-size limit of 500 on the lexical side, plus a small rescue boost that pulls table chunks in when the question looks numeric (`src/4_retrieval/hybrid_search.py`).
4. **Reranking.** A cross-encoder (`bge-reranker-large`, fp16) compresses the fused list to the top eight chunks — the only place where a single component decides what the model will actually see (`src/4_retrieval/post_retrieval.py:104`).
5. **Post-retrieval conditioning.** A table shield validates table chunks, a cylinder reorder redistributes the survivors so one document cannot dominate the context, and the post-retrieval pipeline runs only because its flag is on (the orchestrator forces hybrid mode whenever post-retrieval is enabled).
6. **Context assembly.** Placeholders are resolved back to real tables, duplicates are dropped, everything is packed into a budget of roughly twelve thousand characters (`src/pipeline.py:1961`), and — critically — if the question is about numbers, real table rows are injected alongside the prose so the generator can quote them.
7. **Generation.** The model is called with temperature 0, a fixed seed, JSON-mode structured output, and up to three retries; a deterministic fallback answer, and ultimately a canned safe message, are the last lines of defense (`src/pipeline.py:1992`).
8. **Guardrail.** An asynchronous numerical verifier re-derives every figure it can extract from the model's own evidence block with regexes and compares them against the prose (`src/pipeline.py:2028`); mismatch marks the verdict `failed`. The status flows back into the cache write decision and into enforcement.
9. **Enforcement and delivery.** Depending on configuration, a failed verdict either swaps the answer for a safe fallback or merely annotates it — `GUARDRAIL_ENFORCE=false` by default, so today the layer is advisory. The response is logged to the audit collection, metrics are pushed to MLflow (`src/pipeline.py:2048`), and the run is closed (`src/pipeline.py:2099`).

1.4 A question's life (streaming)

The streaming path is a sibling of the above, not a wrapper around it: it enters the same orchestrator through `query_stream`, but **never reads the cache** — every streamed turn does real retrieval. As tokens arrive, a character-level state machine decodes the JSON answer, emitting only the top-level `answer` field and releasing it on word boundaries so the UI never shows a half-word; citations, refusal flags, and the evidence block are held back until the object closes. The guardrail verdict is buffered until the full answer is known (the D2 pattern), then enforced with the same swap-or-annotate helpers as the sync path before a final `done` event carries sources and status to the browser.

1.5 The improvement loop

Nothing in the hot path tunes itself; quality is improved out of band by the evaluation harness. A synthetic question set (hard-capped at 25 items) is generated from indexed corpora, the batch runner executes it through the pipeline and captures each run's private context list, an LLM judge scores four metrics — faithfulness, answer relevance, context precision, and context recall — against thresholds of 0.85, 0.80, 0.75, and 0.80, and a gate script decides whether the candidate is promoted in the MLflow registry under the `Staging` or `Production` alias. Only after that loop says yes does a code change reach users.

With the map in hand, the rest of this document walks every box on it.

---

2. DETAILED IMPLEMENTATION WALKTHROUGH

Each component below is presented the same way: what it is and what problem it solves, then how it is actually implemented, in the order the code executes. Line anchors are sparse pointers into the real files, not a substitute for them.

2.1 Ingestion — turning a filing into retrievable evidence

Parsers — `app/api/parsers.py`

Objective. Convert whatever a human drops into the upload box — an SGML 10-K, an HTML filing, a PDF, a DOCX report, or a plain text file — into Markdown text plus a list of extracted tables, so that every later stage sees one uniform representation.

How we actually implemented it. The dispatcher inspects the file extension and, for SGML/HTML, runs a purpose-written parser that strips boilerplate tags, decodes HTML entities, and normalizes whitespace while keeping heading structure, because the metadata stage later depends on section headers surviving intact. PDFs take the LlamaParse path — an external parsing service chosen because financial PDFs are dense two-column layouts with merged cells that naive text extraction destroys — with a pypdf fallback that kicks in when the service is unavailable, so ingestion never hard-fails on a network hiccup. DOCX files use the same LlamaParse wrapper, falling back to python-docx — which walks paragraphs and tables separately and re-merges them in document order — when the service is unavailable. Plain text passes through with only newline normalization. The output contract every parser must satisfy is the same: a Markdown string in which tables have already been flattened by the table layer described next.

Table extraction and placeholder planting — `html_table_parser.py`

Objective. Financial questions are usually about table numbers, but tables lose their structure when flattened into prose. The solution is to keep two views of every table: a human-readable flattened Markdown view for the model, and a machine-anchored placeholder that lets later stages recover the original grid.

How we actually implemented it. For each HTML table, the extractor serializes rows and cells into a Markdown pipe table, and at the exact position where that table appeared in the document it plants a `%%TABLE_N%%` token with a monotonically increasing index. The flattened table text is stored alongside the token in a side map. This means the cleaning stage can normalize the surrounding prose without ever touching table content (the `§FIN{n}§` protected notation described next keeps the two worlds apart), the chunker can treat a table as an atomic unit, and — most importantly — during context assembly the placeholder can be swapped back in for the **real** rows of the original grid when the question is numeric. The token format is deliberately grep-friendly: `%%TABLE_` is unique in the corpus and easy to assert against in tests.

Financial text cleaning — `src/1_ingestion/cleaning.py`

Objective. Make prose uniform and machine-checkable without ever changing the value of a number — a cleaner that "tidies" `1,234.56` into `1234.56` inconsistently would corrupt evidence for the guardrail downstream.

How we actually implemented it. The module runs an ordered eight-step pipeline: unicode normalization (smart quotes, non-breaking spaces, dash variants mapped to ASCII), whitespace collapsing, dollar-sign and currency normalization, thousands-separator handling applied only outside protected regions, punctuation tidying, list-marker normalization, and two docstring-level guarantees that matter more than the regexes: no step may alter digit sequences, and no step may delete content between two `§FIN{n}§` markers. Tables extracted earlier are pre-substituted into that protected notation before any step runs and restored afterwards, so a table's `$` and `,` characters are untouchable by design rather than by luck. The stage is idempotent — running it twice is identical to running it once — which matters because the re-ingestion path may clean an already-clean document.

Metadata extraction — `src/1_ingestion/metadata_extractor.py`

Objective. Decide what the document **is** (filing type, company, fiscal year, document type) because retrieval weights and answer phrasing both depend on it.

How we actually implemented it. The classifier walks a table of 20 regex patterns covering the SEC's section vocabulary — `Item 7. Management's Discussion`, `Item 8. Financial Statements`, `Item 1A. Risk Factors`, through to `Part IV` and `Signatures` (`metadata_extractor.py:22`) — over the first portion of the cleaned text. Whichever pattern fires first determines the document type; ticker and company are lifted from accession-number and header conventions. The one deliberate incompleteness: when the accession number is ambiguous, the extractor returns `None` for the document type instead of guessing, because a wrongly-typed document silently pollutes year- and section-filtered retrieval in a way an untyped one does not. **Audit note:** an older second chunk-id generator still exists in this file and would emit different identifiers; it is superseded by the hybrid chunker's `_make_chunk_id`, and the repository's identity tests pin the chunker's format as the only canonical one.

Chunking — `src/1_ingestion/hybrid_chunker.py`

Objective. Split the document into retrieval units that are small enough to embed precisely, big enough to stay self-contained, and — above all — **stably identified**, so that re-ingesting the same filing rewrites the same records instead of accumulating duplicates.

How we actually implemented it. The chunker is tiered by content type. Prose is split on a token budget with sentence-boundary back-off, so a chunk never cuts a sentence in half if it can avoid it. Tables are kept whole whenever they fit the budget and split by row groups otherwise, never mid-row. Each resulting chunk gets the canonical id `ticker_type_md5[:12]_index:04d` (`hybrid_chunker.py:273`), where the md5 hashes the tuple `ticker:fiscal_year:chunk_type:source_file:index` — the id therefore depends on **identity and position within a known document**, not on ingestion order or timestamps, and it changes only when the document itself changes. That single decision is what makes the whole identity layer work — a re-parse of the same filing produces the same ids for unchanged text, which lets the indexer upsert, lets Qdrant replace points by deterministic UUID, and lets `safe_replacement` verify completeness before deleting anything.

Embedding engine — `src/1_ingestion/embedder.py`

Objective. One warm, GPU-resident model that every caller shares, so that a query vector and a document vector are always produced by the same encoder.

How we actually implemented it. The `EmbeddingEngine` loads `nomic-ai/nomic-embed-text-v1.5` (768 dimensions) once at startup and asserts CUDA is available, because silent CPU fallback would halve throughput and, worse, produce vectors from a different execution path than the documented one. Batching is fixed-size with truncation at the model's max length. The query side of the pipeline imports this same engine rather than instantiating its own — the parity between indexed and searched vectors is a property of **sharing one object**, not of configuration discipline, which is why the engine is constructed in the orchestrator's `__init__` and passed down.

Dual-store indexing — `src/1_ingestion/database_indexer.py`

Objective. Persist every chunk twice, in two systems with different strengths: MongoDB for the raw, authoritative text and record metadata; Qdrant for fast approximate neighborhood search.

How we actually implemented it. The write path is transactional in spirit, if not in mechanics: MongoDB is written first with an upsert keyed on the unique `chunk_id` index, so a repeat ingestion updates in place; then Qdrant receives a point whose ID is `uuid5(NAMESPACE_DNS, chunk_id)` — a namespace-qualified UUIDv5, meaning the same chunk always maps to the same point regardless of which worker wrote it. The Qdrant payload is deliberately narrow — seven declared fields (`database_indexer.py:51`): `chunk_id`, `ticker`, `fiscal_year`, `section`, `doc_type`, `contains_table`, and `source_id` — never the raw text. Raw text stays in Mongo and is fetched by id at answer time. This keeps the vector store small, keeps a re-embedding project from duplicating the corpus, and means the payload cannot drift from the database — it only carries routing facts.

Identity controls — `src/1_ingestion/safe_replacement.py`, `reconcile_identity.py`

Objective. Make re-ingestion safe in the failure modes that actually happen: the new parse succeeded but the old chunks were already deleted (data loss), or the two stores quietly disagree about which chunks exist (drift).

How we actually implemented it. `safe_replacement` implements a verify-then-delete dance in six steps: write the new chunks under their ids, verify that every expected id exists in both stores, and only then remove the superseded points — with a rollback flag that aborts deletion if verification fails, leaving the old data intact rather than half-replaced. `reconcile_identity` is the read-only sibling: it walks both stores, computes the symmetric difference of identities, and reports missing-in-Qdrant, missing-in-Mongo, and orphaned points without mutating anything. It exists because auditing drift must never itself be a source of drift. Both are covered by dedicated ingestion-identity tests in

`test/test_ingestion_identity.py`.

Ingestion orchestration — `_ingest_document` in `app/api/main.py`

Objective. Sequence the stages above in one place, with progress reporting, instead of having each endpoint reimplement the order.

How we actually implemented it. The orchestrator receives a parsed payload, runs cleaning, then metadata, then chunking, then embedding, then the dual write, updating a task record after each stage so the upload endpoint can poll progress. Failures at any stage abort with the task marked failed and a message naming the stage — the upload API never reports success for a partially indexed document. There is no background SEC downloader anywhere in this repository; seeding is an operator action (files are copied into the container and then this orchestration is invoked), which is worth stating plainly because the project's own README at times implies automation that does not exist.

2.2 Caching, memory, and pre-retrieval

Redis client — `src/2_caching/redis_client.py`

Objective. One connection strategy shared by the cache, the rate limiter, and the optional job queue, so that Redis connectivity is solved exactly once.

How we actually implemented it. The module exposes a lazily-initialized client with a connection pool, short socket timeouts, and graceful degradation: if Redis is unreachable, the cache returns misses and the rate limiter falls back to allowing traffic, because availability of the answer path matters more than strict throttling. Fail-open semantics here are the reason a Redis outage degrades performance rather than taking the API down.

Semantic cache — `src/2_caching/semantic_cache.py`

Objective. Answer an identical or near-identical question without paying for retrieval and generation, while never serving an answer whose evidence has not been checked.

How we actually implemented it. The cache is Redis-only and three-tiered. Tier one is an exact key: the question is normalized (casefolded, whitespace-collapsed) and hashed with SHA-256, and a hit is a plain dictionary lookup. Tier two compares embeddings by cosine similarity with two thresholds — 0.96 for "same question", 0.60 as a floor below which nothing is considered — and tier three resolves candidates in the ambiguous band by running the cross-encoder reranker and requiring 0.75 or better. Entries carry a TTL that depends on their origin: seven days for questions asked through the static knowledge path, one hour for ad-hoc questions, so stale ad-hoc context expires quickly. Two gates control reads and writes. Reads are refused unless the stored entry's guardrail verdict is literally `passed` — a failed answer can never be replayed from cache. Writes, however, happen **without** embeddings in production: the pipeline's read path passes an embedding function, but the guardrail-adjacent write path does not, so tier-two and tier-three matches never have the vectors they need and **the deployed cache effectively operates on exact-key hits only** (a known limitation, not a design goal). One more asymmetry: the streaming path (`query_stream`) never consults the cache at all, so a streamed turn always does real retrieval.

Conversation memory — `src/5_generation/generator.py` (`ConversationMemory`), `src/pipeline.py` (`_SESSION_MEMORY`)

Objective. Let a follow-up question like "and the second quarter?" resolve against what was just asked, without a full transcript growing without bound.

How we actually implemented it. `ConversationMemory` (`src/5_generation/generator.py:170`) keeps the last `history_k = 3` turns (`config/settings.py:115`) and injects them into the generation prompt as a short dialogue preamble. The API layer additionally maintains a per-session dictionary `_SESSION_MEMORY` that is restored into the pipeline at the start of each request and persisted after the answer is produced, keyed by the caller's session id. Two safeguards matter: the memory is sanitized before it is written back (`_sanitize_session_memory` in `app/api/main.py` strips control characters and truncates runaway turns), and a helper `replace_last_ai` lets the guardrail's post-hoc verdict **rewrite the last assistant turn** — so if enforcement decided the answer was unsafe, memory remembers the safe version, not the original. This is what stops a refused answer from steering the next turn.

Intent router — `src/3_pre_retrieval/intent_router.py`

Objective. Decide, before any retrieval, whether a question is dangerous (and must be halted), whether it needs rewriting/expansion, or is a normal retrieval query — and to do so with one cheap LLM call rather than a chain of them.

How we actually implemented it. A single **combo prompt** asks the model to classify safety, action, ticker, and year and, when useful, to rewrite or expand the query — all in one response, which is why the whole route costs one API call. The primary provider is Groq running `qwen2.5-3b`, chosen for latency; any provider failure falls back to the rule-based

`RuleBasedIntentRouter`, which pattern-matches tickers, years, and known phrasings. A `SECURITY_VIOLATION` verdict short-circuits the pipeline into a fixed refusal string before retrieval is ever attempted. **Audit note:** the whole subsystem is off by default — `ENABLE_PRE_RETRIEVAL=False` — and the setting is not even imported by the orchestrator, so today this code is dormant in production traffic; it is described here at full depth because it is complete, tested code that will be switched on.

Query expansion — `src/3_pre_retrieval/query_expansion.py`

Objective. Improve recall for sparse or jargon-heavy questions by adding plausible terms, without hallucinating new facts into the query.

How we actually implemented it. Expansion is deliberately rule-based rather than generative: synonyms for financial vocabulary, canonicalization of ticker forms, and fiscal-year normalization are applied deterministically, and the result is **unioned** with the original query rather than replacing it — so expansion can add recall but can never discard the user's actual words. This conservatism is the same philosophy as the rest of the pre-retrieval stage: the risky, model-driven path (the combo prompt) is optional, and the deterministic path is what runs when anything is uncertain.

Legacy coreference resolution — `src/3_pre_retrieval/` coreference module

Objective. Resolve pronouns ("its revenue", "the company") to a concrete entity so retrieval searches for a company rather than a pronoun.

How we actually implemented it. The resolver is a light pattern-and-NER pass that anchors pronouns to the nearest previously named entity or ticker in the session, without a heavyweight coreference model. It runs only on the legacy route and, like the rest of pre-retrieval, is bypassed by default. Its practical value today is in tests and in the fallback router, where pronoun-stripped queries measurably improve the rule matches.

2.3 Retrieval and post-retrieval

Hybrid search — `src/4_retrieval/hybrid_search.py`

Objective. Combine semantic similarity (which understands paraphrase but misses exact tickers and figures) with lexical matching (which nails exact terms but misses paraphrase) so that neither failure mode alone decides what the model sees.

How we actually implemented it. The dense leg queries Qdrant for the top candidates by cosine distance over the 768-dimensional embeddings. The lexical leg runs BM25 over a corpus view limited to 500 documents, gated by a token-overlap test — if the query shares almost no content words with the corpus, BM25 would return noise, so it is skipped rather than allowed to poison the fusion. The two ranked lists are then merged with Reciprocal Rank Fusion at `k = 60`: each document's fused score is the sum of `1/(60 + rank)` across the lists it appears in, which needs no score calibration between an LLM-derived cosine and a term-frequency statistic. On top of fusion sits a rescue rule: when the question looks numeric (contains a figure, a percentage, or a financial keyword), table-typed chunks get a 0.5 boost, so a table that ranked just below the cut is pulled in — numbers live in tables, and pure semantic rank often buries them. The module also supports multi-entity queries by fusing per-ticker result sets, which the balancing stage below consumes.

Dense-only retrieval — same module, non-hybrid path

Objective. Provide a working retrieval path when hybrid mode is switched off (or when BM25's corpus view is unavailable), so the system degrades to **worse recall** rather than to **no answer**.

How we actually implemented it. With hybrid disabled, only the Qdrant leg runs, and an enrichment pass re-ranks by query-term overlap to recover some of what lexical search would have contributed. The orchestrator, however, treats hybrid as effectively mandatory in the default configuration: `ENABLE_HYBRID_RETRIEVAL=True` is the shipped value and the post-retrieval flag **forces** hybrid on even if it was turned off, because reranking, shielding, and reordering all assume a fused list to work with. That coupling is intentional flag-stacking rather than an accident — the stages were written as one unit.

Cross-encoder reranking — `src/4_retrieval/reranker.py`

Objective. Replace the cheap "is this chunk near the query?" judgment with an expensive "given this exact pair, does this chunk actually answer this exact question?" judgment, on a shortlist only.

How we actually implemented it. `bge-reranker-large` scores each (query, chunk) pair with a cross-encoder on fp16 CUDA — fp16 because the model is score-comparable across precision levels and fp16 roughly doubles throughput — and the top `top_n = 8` chunks survive. Eight is chosen against the context budget: eight chunks plus injected tables fit

inside the ~12k-character window with room for the prompt. The reranker is the single most important precision control in the pipeline; everything after it is conditioning rather than selection.

Table shield — `src/4_retrieval/table_shield.py`

Objective. Validate that table chunks entering the context are structurally sound — well-formed rows, aligned columns — and quarantine malformed ones.

How we actually implemented it. The shield parses each table chunk back into rows, checks column-count consistency and header presence, and flags violations. **Audit note:** in the shipped configuration the shield is instantiated with `clean_enabled=False`, which makes it a pass-through no-op in the default pipeline — the validation code exists and is tested, but production currently does not enforce it. The orchestrator calls it unconditionally, so flipping the flag activates real filtering with no other change.

Cylinder reorder — `src/4_retrieval/cylinder_reorder.py`

Objective. Rerank-favoritism problem: a cross-encoder tends to cluster all chunks from the same filing next to each other, so the context becomes eight passages of one document. The reorder spreads sources across the ranking so the model sees multiple perspectives.

How we actually implemented it. The fixed permutation `(0,2,4,6,7,5,3,1)` maps positions so that items from distinct source groups interleave — position 0 keeps the best chunk, positions 1 and 2 draw from far-apart slots, and so on. When the ranked list is shorter than the permutation, the pattern degrades gracefully to the original order. The name comes from the shape of the index mapping (a closed loop through the list). It is a cheap deterministic step — no model, no I/O — placed **after** reranking so it reshuffles the survivors rather than the candidates.

Multi-ticker balancing — `src/pipeline.py` (retrieval sub-routines)

Objective. Stop the single best-matching ticker from crowding out a second company the question also names, which is what plain hybrid retrieval does when one company's filings dominate the corpus.

How we actually implemented it. When the query resolves to more than one ticker, the orchestrator issues a **separate retrieval per ticker** (sequentially in the sync path, in batches in the streaming path), then interleaves the per-ticker top chunks into one balanced list — each company guaranteed representation proportional to the allotment rather than to corpus size. A cross-ticker bypass exists for questions that only appear multi-entity: if after routing only one ticker is real, the balancing code steps aside and the single-ticker path runs, because guaranteeing representation for a company the user did not ask about would inject irrelevant context. The batching variant in the streaming path exists purely for latency — one round trip instead of N.

Post-retrieval pipeline — `src/4_retrieval/post_retrieval.py`

Objective. Run the conditioning trio (shield → reorder → final trimming) in a fixed order with one entry point, rather than having the orchestrator choreograph three modules by hand.

How we actually implemented it. The module exposes a single function that takes the reranked list, applies the table shield, applies the cylinder reorder, enforces the final top-N cut, and returns the conditioned list. Because this stage is flag-gated (`ENABLE_POST_RETRIEVAL=True` by default), its presence is also what forces hybrid mode and reranking upstream — the orchestrator's flag resolution treats the three features as one deployable unit. The entry point is where the pipeline's flag coupling is most visible: turning off post-retrieval silently downgrades retrieval to the dense-only path unless hybrid is reasserted independently.

2.4 Context assembly and generation

Placeholder resolution and context budgets — `src/pipeline.py`

Objective. Turn the conditioned chunk list into the exact string the model will read: real tables restored, duplicates removed, everything inside a hard character budget.

How we actually implemented it. The assembly runs in a deliberate sequence. First, `%%TABLE_N%%` placeholders are resolved against the side map, so any table the retrieval kept as a token is expanded back into real rows. Second, deduplication drops chunks that share an identity with something already admitted — identity here is the canonical `chunk_id`, not text similarity, so near-duplicate prose from overlapping chunks is caught only if the chunker produced identical ids (it does not), and the honest statement is that dedup is **exact-identity** dedup. Third, chunks are packed greedily until the total context would exceed roughly 12,000 characters; when a table would be dropped by the budget, the code prefers keeping a truncated table over dropping it entirely, because a partial table still carries numbers while a missing one carries nothing. Fourth, **query-aware table injection**: if the question is numeric, actual table rows matching the query's entity and figure are injected from the table side map even when retrieval did not surface them —

this is the pipeline's answer to "the model needs the real number, not a prose summary of the number." Finally, any residual marker notation (`§FIN{n}§` remnants and stray placeholders) is stripped so the model never sees internal syntax. The budget is enforced as a total (prompt + context + history), not per-chunk, so long history cannot silently evict evidence.

Generation — `src/5_generation/generator.py`

Objective. Produce a structured, citation-bearing financial answer whose every number comes from the provided context — under settings that make the model maximally reproducible.

How we actually implemented it. The call runs `gpt-oss-120b` (with `gpt-oss-20b` as the configured alternate) at temperature 0 with seed 42 and `max_tokens = 2048`, temperature-zero because the guardrail compares output numbers to evidence and a nondeterministic sampler would make that comparison flaky across retries. The model is asked for JSON directly (JSON mode), with the schema constraints expressed in the prompt: an `answer`, a `citations` array, an `internal_thought` field the validators require to contain a digit (forcing the model to actually touch figures), and an `extracted_raw_data` block the guardrail will re-derive. Retry policy is three attempts with backoff, then a deterministic fallback answer assembled from retrieved context without model generation, then — if even that fails — a canned safe message. Every layer is designed so that a provider outage degrades the answer to "less fluent" rather than to "empty or fabricated."

Answer schemas and refusal classification — `src/5_generation/schemas.py`

Objective. Make "is this answer acceptable?" a type-level question answered by validators, not a convention enforced by hope.

How we actually implemented it. `ConsolidatedFinancialAnswer` is a pydantic model whose validator enforces the core invariant: unless the answer is classified as a refusal, it must carry at least one citation; every citation must resolve to a real source id; and the internal thought must contain a digit. `RefusalAnswer` models the honest negative — the structured way to say "I don't have that" with a reason code. `classify_generation_output` splits raw model text into one of those two shapes (or a failure), handling the case where the model wrapped JSON in prose or fences by extracting the outermost object with a bracket-depth scan. Because validation runs on **every** answer before it reaches the enforcement layer, a citation-free non-refusal never leaves the generation module — the invariant is checked at the boundary where it can still be repaired by retry.

Streaming JSON state machine — `src/5_generation/streaming_json.py`

Objective. Stream a JSON answer token-by-token to the browser while still guaranteeing that only the `answer` field — validated, whole-word — is shown live.

How we actually implemented it. The extractor is a character-level state machine that tracks whether it is inside the top-level `answer` string (depth-1 object keys only), string escapes, and nested values. Chunks arriving from the provider are fed in as raw characters; whenever the accumulated answer text ends on a word boundary, that prefix is released as a delta and removed from the buffer, so the UI never renders a half-word like `rev` mid-stream. Nested structures (citations, raw data) are buffered silently until the closing brace, then handed to the same schema validators as the sync path. On malformed input the machine fails safe: it stops emitting and lets the caller fall back to the non-streaming answer, rather than streaming a corrupted fragment.

Guardrail — numerical verification — `src/5_generation/async_guardrail.py`

Objective. Verify, after generation but before delivery, that every number the model asserted can be re-derived from the evidence the model was given.

How we actually implemented it. The guardrail extracts every number-like token from the generated answer, extracts every number-like token from the model's own `extracted_raw_data` block (the facts the model claims it pulled from context), and requires each answer figure to appear in that evidence set — a pure regex membership test with no LLM call, run in a thread pool so it never blocks the generation path. Comparison normalizes both sides first: currency symbols, `%`, `M/B/K` suffixes, commas, and whitespace are stripped, leading zeros are tolerated on either side, and a trailing punctuation bug fixed as the D8 correction (`rstrip(".,")` applied so a token absorbed from `2025.` compares equal to `2025`) is removed. An answer containing no numbers passes by definition. The verdict is one of three statuses: `passed` (all figures consistent), `failed` (a figure could not be grounded), or `error` (the guardrail itself raised — treated as `passed=False` for safety but recorded distinctly so a broken guardrail is visible rather than silent). The async path runs under a 5-second timeout that converts slow verification into `error` rather than stalling the response. A `SAFE_FALLBACK` response type exists for the `failed` case so the caller always has something valid to show. **Audit note:** because the evidence set is the model's own extraction rather than the raw context, the check verifies answer-versus-extraction consistency — a model that misquotes its own extraction is caught, while an error that originates inside `extracted_raw_data` itself is only bounded by the schema validators upstream.

Guardrail enforcement — `app/api/main.py`

Objective. Decide what the **user** sees when the verdict is not `passed`, with policy that is configurable per deployment.

How we actually implemented it. Four small helpers carry the policy: `_validate_guardrail_config` (rejects nonsense flag combinations at startup), `_guardrail_should_swap` (should the answer be replaced by the safe fallback?), `_guardrail_should_withhold` (should it be withheld entirely?), and `_apply_guardrail_enforcement` (performs the swap or annotation on the response payload). Under `GUARDRAIL_ENFORCE=false` — the shipped default — enforcement is advisory: the verdict is computed, logged, propagated into the cache-write decision (a `failed` answer is never cached), and attached to the response metadata, but the original answer is still delivered. Two further helpers matter for correctness: `_sanitize_session_memory` keeps garbage out of stored history, and in the streaming path a D2-style buffering step holds the answer until the guardrail has seen it whole, so a streamed user never watches an answer that is about to be swapped. Toggling enforcement to `true` requires no code change — the helpers are the single choke point.

2.5 Orchestration — `src/pipeline.py`

Pipeline construction — `FinancialRAGPipeline.__init__`

Objective. Assemble all ten stages into one object with their flags resolved consistently, once, at startup — so a request never pays construction cost and never encounters contradictory feature combinations.

How we actually implemented it. The constructor builds the components in dependency order — embedding engine first (everything downstream takes it), then ingestion helpers, cache, pre-retrieval modules, retrieval modules, reranker, guardrail, and generator — each conditionally constructed behind its own feature flag but **always** constructed as an attribute when its flag is on, so later stages can reference without null checks. Flag resolution happens here too and it is deliberately opinionated: enabling post-retrieval forces hybrid retrieval on (the conditioning stages need a fused list), and disabling everything optional still leaves generation and guardrail mandatory — there is no configuration in which the system answers without a verdict. Configuration comes from `config/settings.py`, read once; nothing in the hot path re-reads environment variables.

Synchronous query — `query()`

Objective. Execute the full hot path — run open, cache, routing, retrieval, conditioning, assembly, generation, guardrail, metrics, run close — with every stage observable and every failure contained.

How we actually implemented it. The method opens an MLflow run named after the request so all later metric writes land in one place, then proceeds strictly in order: the cache is consulted only when the prior verdict permitted caching; the pre-retrieval route (dormant by default) may rewrite, expand, or halt the query; hybrid retrieval produces the candidate list which reranking cuts to eight, after which shield, reorder, and balancing condition it; an empty result short-circuits to a structured refusal rather than generating over nothing (one of several explicit early exits — a security halt and a cache hit being the others); context assembly then builds the prompt; the generator runs with its retry ladder; the guardrail runs and its verdict is attached; metrics (latency per stage, cache status, guardrail outcome) are written to the open run; and the run is closed. Each stage's output is carried in a single request-scoped state object rather than passed positionally, which is why adding a stage never changes a neighbor's signature — the pattern that has kept a 2,400-line orchestrator readable.

Streaming query — `query_stream()`

Objective. The same pipeline with token-level delivery, plus the two honest differences from sync mode: no cache read, and guardrail enforcement after the fact.

How we actually implemented it. The method mirrors `query()` through retrieval and assembly but skips the cache lookup entirely (every streamed turn is a real turn), then hands the generation call to the streaming path: provider chunks flow through the JSON state machine, which emits `token` deltas for the answer field, then a final structured `answer` event once the object closes. Because the full answer is only known at that point, the guardrail runs **after** streaming completes and its verdict rides out in a `done` event alongside sources and status — with enforcement applied to that final event (the D2 buffering pattern), so a swapped answer replaces the tail the client was about to commit to. Sub-retrievals for multi-ticker questions are batched here rather than sequential to keep time-to-first-token down. The method ends by closing its MLflow run the same way as `query()`, so streamed and non-streamed traffic are comparable in tracking.

2.6 Evaluation — `evaluation/` (repository root)

Synthetic dataset generation — `evaluation/synthetic_generator.py`

Objective. Create a ground-truth question set from the indexed filings themselves, so evaluation measures the system against questions its own corpus can answer.

How we actually implemented it. The generator samples chunks from the corpus and asks an LLM to write question–answer pairs grounded in those chunks, then validates each pair against the source text before admitting it — an ungrounded synthetic question would reward hallucination. Generation is hard-capped at 25 items, which is a deliberate ceiling: the set is meant for regression gating in minutes, not benchmarking in hours, and the cap keeps judge costs bounded. The output is a frozen JSON artifact that every later stage consumes, so regeneration never silently changes historical scores.

Batch runner — `evaluation/batch_runner.py`

Objective. Run the whole evaluation set through the real pipeline and capture what the model actually saw — because judging an answer without its context cannot distinguish a bad generation from bad retrieval.

How we actually implemented it. For each question the runner invokes the pipeline in the same way a user turn would, then reads the pipeline's private `_last_contexts` attribute to record the exact chunks that were retrieved. That dependency on a private field is the runner's known architectural debt — it couples evaluation to orchestrator internals — but it was chosen over logging contexts separately, which risks capturing a **different** context than the one the answer used. The result is a trace file holding question, answer, retrieved context, and metadata for each item, which is what the judge consumes.

LLM-as-judge — `evaluation/judge_evaluator.py`

Objective. Score each trace on the four properties that define RAG quality, with a judge prompt stable enough to gate releases on.

How we actually implemented it. The judge scores faithfulness (is every claim in the answer supported by the retrieved context?), answer relevance (does it address the question?), context precision (was the retrieved set focused?), and context recall (did it contain what was needed?) — each on a 0–1 scale, returned under the metric names `faithfulness`, `answer_relevance`, `context_precision`, and `context_recall` (`judge_evaluator.py:48`). Gates are applied downstream at thresholds of 0.85, 0.80, 0.75, and 0.80 respectively (`mlflow_tracker.py:8`), chosen in that order of strictness because faithfulness is the safety property and recall the most forgiving. Scoring prompts are fixed-shot with rubric anchors to reduce judge drift, and each item's four scores are averaged into the run's aggregate. **Audit note:** no retrieval-ranking metric (MRR, nDCG) is computed anywhere — ranking quality is currently observable only through the indirect lens of context precision.

Quality gate and registry transition — `evaluation/` gate + stage scripts

Objective. Turn scores into a binary release decision, and the release decision into an MLflow registry state — no manual "we think it's better" promotions.

How we actually implemented it. The gate script reads the judge artifact, compares each metric against its threshold, and emits pass/fail per metric plus an overall verdict; on pass it moves the registered candidate under the `Staging` alias, and a second, stricter run promotes it to `Production`. The two-alias scheme means production traffic can be pointed at `Production` while candidates sit at `Staging`, giving an explicit rollback position. The staged helpers (`run_stage_1b`, `run_stage_1c`, `run_stage_1c_resilient`, `run_stage_1d`) sequence generation, judging, and gate execution with resume semantics, so a failed judge call does not restart the batch. **Audit note:** an `artifacts/evaluation_scores.json` exists whose keys no current script writes — evidence of an earlier schema, retained but unreferenced; treat the judge evaluator's own output as authoritative.

2.7 Observability — MLflow, audit log, analytics, health

Experiment tracking — MLflow integration in `src/pipeline.py` and `evaluation/mlflow_tracker.py`

Objective. Every question, every evaluation run, and every promotion decision leaves a machine-readable trail tied to code, data, and configuration.

How we actually implemented it. The orchestrator opens a run per query and logs per-stage latencies, cache hit status, guardrail outcome, and answer metadata; the evaluator logs its four metric scores per run. The tracker additionally maintains registry aliases (`Staging`, `Production`) so "which artifact is live" is always answerable from MLflow rather than from deployment folklore. Runs are cheap and unconditional — the cost of tracking was accepted rather than optimized away, because the audit trail is part of the product's trust story.

Audit logging — `app/api/db_logger.py`

Objective. A durable, queryable record of who asked what, what was returned, and what the guardrail said — independent of MLflow's experiment semantics.

How we actually implemented it. Each completed interaction is appended to the `rag_audit_logs` MongoDB collection with the question, session/user identity, the delivered answer, verdict status, latency, and timestamps. Writes are fire-and-forget from the response path: an audit insert failure must never fail a user's request, so errors are swallowed after logging. **Audit note:** the collection is created without a TTL and without a supporting index — an operational gap worth closing before long-running deployments, since unbounded growth will eventually make admin queries slow.

Analytics — `app/api/analytics.py`

Objective. Answer operational questions — how often does the guardrail fire? which tickers get asked about? what is the latency profile? — from the audit collection.

How we actually implemented it. The summary endpoint aggregates the audit log and computes guardrail pass rates over **eligible** entries only (entries where a verdict was actually produced — dividing by all traffic would dilute the rate with cache hits that never ran the guardrail). Tertile bucketing places questions into three groups using 0.66/0.33 split points to build the "easy/medium/hard" distribution view. **Audit note:** the frontend's mock dashboard labels these bands at 0.8/0.5 — a mismatch between what the UI displays and what the API computes, visible whenever the real analytics endpoint is wired to the dashboard.

Health — `app/api/main.py` health endpoint

Objective. One unauthenticated, always-available signal of whether the stack's dependencies are alive, for Docker healthchecks and operators.

How we actually implemented it. The endpoint probes MongoDB, Qdrant, and Redis with short timeouts and reports per-dependency status plus process-level facts (uptime, pipeline warm state). It is deliberately excluded from rate limiting so a monitor polling every second is never throttled into false alarms, and it never touches the LLM providers — an external Groq outage should not flip the container unhealthy.

2.8 Backend API and async layer — `app/api/` (70% depth)

API surface and middleware — `app/api/main.py`

Objective. Expose the pipeline and the ingestion machinery over HTTP with consistent errors, CORS, request sizing, and rate limits — in one application module that a single process serves.

How we actually implemented it. The FastAPI app is assembled at module level with middleware applied outermost-first: CORS restricted to the configured frontend origin, body-size limits so a hostile upload cannot exhaust memory, and the rate-limit dependency injected per-route rather than globally (routes that must never be throttled — health, authentication — simply opt out). The app registers ten pipeline/data endpoints (chat, streaming chat, file upload, task status, cache delete, health, analytics, audit logs, database clear, ingest) and nine auth routes. Error handling is centralized: a small set of exception handlers maps domain failures (unauthorized, forbidden, not found, bad input, pipeline failure) onto stable JSON error bodies, so the frontend never has to parse FastAPI's default debug shapes. Startup is handled by a lifespan context that warms the model loader — constructing the embedding engine and pipeline before the first request — so the first user turn does not pay model-load latency.

Authentication — `app/api/auth/`

Objective. Issue and verify short-lived access tokens with longer-lived refresh tokens, plus credential registration and role management for a small operator set.

How we actually implemented it. Passwords are hashed with bcrypt via pwdlib; login verifies credentials and returns an HS256 access token with a 30-minute lifetime alongside a 7-day refresh token. Refresh issues a new access token but does **not** rotate the refresh token — a known gap: a stolen refresh token remains valid for its full week. A blacklist exists for revoked tokens, but verification treats blacklist-store failure as **allow** (fail-open), trading strict revocation for availability — deliberate, and worth knowing before a compliance conversation. Roles (`superadmin`, `admin`, `analyst`, `user`) are seeded for the superadmin account at startup if no such account exists, so first boot always has a way in.

Authorization and the `get_current_user` collision — `app/api/main.py`, `app/api/auth/`

Objective. Enforce role-based access on privileged routes while keeping chat usable for anonymous or degraded-auth callers.

How we actually implemented it. `require_roles(...)` is the enforcing dependency: it resolves the token, loads the user, and returns 403 "Insufficient privileges" when the role is not permitted — guarding upload, ingest, cache-delete, database-clear, analytics, and audit-log routes. **Audit note:** two different functions are named `get_current_user` — the app imports the rate-limiter-aware variant from `app/api/rate_limiter`, which **tolerates** anonymous callers, and only the `require_roles` endpoints ever strictly enforce JWT validity. The practical consequence: the chat endpoint works without a token (subject to IP rate limiting), while privileged routes still demand full authentication. Reading `main.py` without noticing the import shadowing leads to the wrong conclusion about which endpoints are protected.

Rate limiting — `app/api/rate_limiter.py`

Objective. Fixed-window request budgets per user or per client IP, backed by Redis so limits hold across workers.

How we actually implemented it. On each request the limiter resolves a key — `user:<id>` when a token identifies the caller, `ip:<address>` otherwise — then increments a Redis counter for the current window and rejects with 429 when the configured budget is exhausted (chat ships at 10/minute; overall tokens come from `RAG_API_TOKENS`). Window reset is time-based rather than sliding, which permits brief boundary bursts — accepted simplification. When Redis is unreachable the limiter fails open, consistent with the rest of the availability-over-strictness posture.

Chat and streaming endpoints — `app/api/main.py`

Objective. The two user-facing routes: one returns a complete answer, the other an SSE stream, both restoring conversation context before the pipeline runs.

How we actually implemented it. Before invoking the pipeline the sync handler calls `_restore_session_memory` to load the session's stored history into the pipeline's memory object, and after the answer returns it persists the updated turns — which is what makes multi-turn follow-ups work across separate HTTP requests. The guardrail enforcement helpers then apply the configured swap/annotate policy, and the response is logged through the audit logger before being returned. The streaming route wraps `query_stream`, sets SSE headers, and translates the pipeline's event vocabulary (token, answer, sources, done) into the wire format the browser parses, applying the same D2 buffering so enforcement decides before the final event is committed. Both routes are rate-limited per the budgets above.

Upload, ingest, and the task registry — `app/api/main.py`, `app/api/worker.py`

Objective. Accept a document, index it, and let the caller poll for progress — without blocking the request thread for the duration of a parse-and-embed cycle.

How we actually implemented it. The upload endpoint (admin-only) stores the file, registers a task record (id, status, stage, error), and schedules `_ingest_document` as a FastAPI `BackgroundTasks` job — the default execution mode. The task-status endpoint returns the record, so the frontend can show stage-by-stage progress. An opt-in path exists for real queue execution: setting `USE_ARQ_QUEUE=true` routes the same job through an ARQ worker against Redis instead — but the shipped `docker-compose.yml` leaves the `arq-worker` service commented out, so the default deployment runs background tasks in the API process. Two admin endpoints complete the data-plane: cache-delete (evict a query key) and database-clear (drop collections), both behind `require_roles`.

Background workers — `app/api/worker.py`

Objective. Keep job execution pluggable — in-process by default, queue-backed when scale demands it — without changing endpoints.

How we actually implemented it. The worker module abstracts the submission decision: by default it hands jobs to FastAPI's `BackgroundTasks` (zero infrastructure, but jobs die with the process — an accepted trade-off for a single-container deployment); with ARQ configured it enqueues to Redis and the separate worker container consumes, giving durability and horizontal scale. The choice is one environment flag, and both paths share the same task-record contract, so polling code is mode-agnostic.

---

3. DOCKER & FRONTEND OVERVIEW

This section is deliberately lightweight — a map of how the pieces communicate, not a second walkthrough.

3.1 Orchestration and networks

The stack is defined in `docker-compose.yml` and consists of seven active services: the FastAPI `backend`, the Next.js `frontend`, the Streamlit operator console, and the stateful trio `mongo`, `qdrant`, and `redis`, plus `mlflow` for tracking. Two networks separate concerns: `frontend-net` carries only browser ↔ backend traffic, while `backend-net` is the private fabric on which the backend talks to its datastores — the browser can never address Mongo, Qdrant, or Redis directly, which is the single most important security property of the topology. The backend is pulled onto both networks. GPU access is declared as a device reservation so the embedding and reranking models run on CUDA inside the container; the image is built from `app/Dockerfile` as a non-root user with the torch CUDA 13.0 wheel set baked in. Volumes persist Mongo data, Qdrant storage (including the test collection checked into `data/qdrant_db_test/`), and MLflow artifacts across restarts. The `arq-worker` service exists in the file but is commented out — the default deployment executes background ingestion in-process, as described in §2.8. Seeding filings is manual: `docker compose cp` places files into the backend container and the ingest endpoint indexes them.

3.2 Frontend communication model — `frontend/`

The browser app is a Next.js 15 App Router project with three routes (chat, dashboard, admin) implemented as tab views over shared components, styled with Tailwind v4 and shadcn primitives. All backend communication flows through a single fetch wrapper, `frontend/src/lib/api.ts`, which owns token storage, attaches the access token to each request, and implements single-flight refresh: when a request returns 401, exactly one refresh call is issued and concurrent failures await it rather than stampeding the auth endpoint. Chat uses the streaming client `frontend/src/lib/ragStream.ts`, which opens the SSE endpoint and parses the pipeline's event vocabulary — `token` events append to the answer area, `sources` renders citations, and `done` finalizes status and guardrail annotations. Three known gaps are worth stating: guardrail verdicts are not yet rendered in the UI (the data arrives, the display does not use it), 429 responses from the rate limiter are not handled specially (a throttled user sees a generic failure), and the dashboard's analytics values are partly mock data with tertile labels (0.8/0.5) that do not match the API's computed split points (0.66/0.33). The Next.js middleware performs an admin check for the admin route, but it is currently dormant in practice — real enforcement for admin operations lives server-side in the API's `require_roles` dependencies, which is where it belongs.

3.3 Streamlit console — `app/ui/streamlit_app.py`

A secondary operator console (chat + status views) lives alongside the main app as a compose service and talks to the same API endpoints; it is a convenience shell for demos, not a second product surface.

3.4 Test suite (for orientation)

The repository carries roughly 650 tests across 39 files under `test/`, covering each pipeline stage plus API, guardrail, streaming-enforcement, and ingestion-identity behavior; there is no pytest configuration file, so they are run explicitly with `PYTHONPATH=app` from the project root (e.g. `python -m pytest test/ -p no:cacheprovider -q`). The most recent full run recorded 662 passed, 1 failed, 0 errors — the failing case being an environment-sensitive assertion rather than a functional regression.

End of document. Every claim above was verified against the code during a full repository scan; where narrative and code disagreed, the code — and an Audit note — won.